StreetRace Manager
Integration Testing Report

This report documents the design, implementation, and integration testing of the StreetRace Manager system.

In simple terms, the goal of this part is to prove that the modules work together correctly, not just that each module works by itself.

0 Contents

1 Requirement Coverage **2**

2 Call Graph (Logic Design) **2**
2.1 Core Function Call Map (20 Inter-Module Arrows) 2

3 Integration Test Design **2**
3.1 Group A: Registration ↔ Crew (6 tests) 3
3.2 Group B: Crew ↔ Race (8 tests) 3
3.3 Group C: Race ↔ Results ↔ Inventory — Financial Sync (7 tests) 4
3.4 Group D: Results ↔ Damage State (4 tests) 4
3.5 Group E: Results ↔ Trophy Room (5 tests) 5
3.6 Group F: Missions ↔ Crew Role Check (8 tests) 5
3.7 Group G: Missions ↔ Inventory Damage (5 tests) 6
3.8 Group H: Tuning ↔ Crew + Inventory (8 tests) 7
3.9 Group I: Sponsors ↔ Inventory (4 tests) 7
3.10 Group J: Full Workflow Integration (10 tests) 8

4 Additional Modules (Bonus) **8**

5 Testing Results Summary **9**
5.1 Bugs Detected & Fixed During Integration 9

6 Exhaustive Integration Audit (Final Proof) **9**
6.1 Audit Lifecycle Phases 9

1 Requirement Coverage

This section maps the assignment requirements to the test groups used in this report.

Table 1. Requirement Coverage Mapping

Assignment Requirement	Covered by Test Groups
Register a driver and then enter the driver into a race	Group A and Group B
Attempt to enter a race without a registered driver	Group A and Group B
Complete a race and verify results plus prize money update the inventory	Group C, Group D, and Group E
Assign a mission and ensure the correct crew roles are validated	Group F and Group G
Verify tuning, sponsorship, and trophy interactions	Group H, Group I, and Group J
Check full end-to-end module interaction across the controller	Group J

2 Call Graph (Logic Design)

The system is designed with a **Controller-Service** architecture. The `StreetRaceManager` acts as the central hub (Controller), invoking methods across **9 specialized modules** (Services). The complete node-by-node atlas is in `diagrams/call_graph_atlas.md`.

2.1 Core Function Call Map (20 Inter-Module Arrows)

- **Race Flow**: `M1: run_race_sequence()` → `R1: create_race()` (calls `C2: is_role()` + `I4: has_car()`) → `RE1: finalize_race()` (calls `R2: get_race()`, `I1: add_cash()`, `I7: set_damage()`, `TR1: add_trophy()`) → `S2: trigger_win_bonus()` (calls `I1: add_cash()`).
- **Tuning Flow**: `M2: perform_tuning()` → `T1: upgrade_car()` (calls `C2: is_role()`, `I6: use_parts()`, `I2: deduct_cash()`, and on failure: `I5: add_parts()` rollback).
- **Mission Flow**: `M3: start_mission()` → `MS1: assign_mission()` (calls `C2: is_role()`, `I8: is_damaged()`, `I7: set_damage()`).
- **Registration Guard**: `C1: assign_role()` → `RG2: is_registered()`.

(Note: The physical Call Graph diagram is hand-drawn and submitted as an image in `diagrams/`.)

3 Integration Test Design

The following **65 test cases** validate how different modules interact with one another. Each test targets a specific inter-module edge from the Call Graph.

3.1 Group A: Registration ↔ Crew (6 tests)

Call Graph Edge: `C1: assign_role()` → `RG2: is_registered()`

Table 2. Group A — Registration ↔ Crew

ID	Scenario	Modules	Expected	Actual	Errors
A01	Assign role to unregistered member	Reg → Crew	Rejected: "not in the system"	Rejected	None
A02	Register driver, then assign driver role (skill=8)	Reg → Crew	Accepted, skill stored as 8	Accepted, skill=8	None
A03	Register mechanic, then assign mechanic role	Reg → Crew	Accepted	Accepted	None
A04	Register strategist, then assign strategist role	Reg → Crew	Accepted	Accepted	None
A05	Assign invalid role (e.g., "hacker") to registered member	Reg → Crew	Rejected: "Invalid role"	Rejected	None
A06	Register same member twice	Reg	Rejected: "already registered"	Rejected	None

Why these tests matter: The assignment explicitly states *"A crew member must be registered before a role can be assigned."* These 6 tests verify the Registration → Crew data gate from every angle: valid roles, invalid roles, unregistered members, and duplicates.

3.2 Group B: Crew ↔ Race (8 tests)

Call Graph Edge: `R1: create_race()` → `C2: is_role()` + `I4: has_car()`

Table 3. Group B — Crew ↔ Race

ID	Scenario	Modules	Expected	Actual	Errors
B01	Valid driver + valid car → race	Crew, Inv → Race	Race scheduled	Scheduled	None
B02	Mechanic tries to race	Crew → Race	Rejected: not a driver	Rejected	None
B03	Strategist tries to race	Crew → Race	Rejected: not a driver	Rejected	None
B04	Completely unregistered person races	Crew → Race	Rejected	Rejected	None
B05	Valid driver, missing car	Crew, Inv → Race	Rejected: car not in inventory	Rejected	None
B06	Both driver and car missing	Crew, Inv → Race	Rejected (first failure)	Rejected	None
B07	Same driver enters two different races	Crew → Race	Both accepted	Both accepted	None
B08	Same car reused across two races	Inv → Race	Both accepted (no lock-out)	Both accepted	None

Why these tests matter: The assignment states *"Only crew members with the driver role may be entered in a race."* These 8 tests verify the Race module correctly delegates role-checking to the Crew module and car-checking to the Inventory module.

3.3 Group C: Race ↔ Results ↔ Inventory — Financial Sync (7 tests)

Call Graph Edges: `RE1: finalize_race()` → `R2: get_race()` + `I1: add_cash()`

Table 4. Group C — Race ↔ Results ↔ Inventory (Financial Sync)

ID	Scenario	Modules	Expected	Actual	Errors
C01	Finalize an existing race	Race → Results	Success	Success	None
C02	Finalize a race that doesn't exist	Race → Results	Rejected: "not found"	Rejected	None
C03	Win $7,500 prize → cash updates	Results → Inv	Cash +$7,500	$10,000→$17	None
C04	Win $0 prize → no cash change	Results → Inv	Cash unchanged	$10,000→$10	None
C05	5 races → cash accumulates correctly	Results → Inv	Cash +$5,000 total	$10,000→$15	None
C06	Race status updates to "Completed"	Results → Race	Status="Completec	Status="Comple	None
C07	Result history stores correct driver/car	Results → Race	Driver="Alice", Car="Supra"	Correct	None

Why these tests matter: The assignment states *"Race results should update the cash balance in the Inventory."* These 7 tests verify the financial data flows correctly from Results into Inventory, including edge cases like zero prizes and batch processing.

3.4 Group D: Results ↔ Damage State (4 tests)

Call Graph Edge: `RE1: finalize_race()` → `I7: set_damage()`

Table 5. Group D — Results ↔ Damage State

ID	Scenario	Modules	Expected	Actual	Errors
D01	Race with damaged=True → car flagged	Results → Inv	is_damaged=True	True	None
D02	Race without damage → car stays OK	Results → Inv	is_damaged=False	False	None
D03	Two cars in garage, only raced car damaged	Results → Inv	Car1=damaged, Car2=OK	Correct	None
D04	Damage persists through other operations	Results → Inv	Still damaged after adds	Persists	None

Why these tests matter: Damage state must persist until a Repair mission fixes it. These 4 tests verify that the Results module correctly communicates damage to the Inventory and that only the specific raced car is affected.

3.5 Group E: Results ↔ Trophy Room (5 tests)

Call Graph Edge: `RE1: finalize_race()` → `TR1: add_trophy()`

Table 6. Group E — Results ↔ Trophy Room

ID	Scenario	Modules	Expected	Actual	Errors
E01	1st place → trophy unlocked	Results → Trophy	has_trophy=True	True	None
E02	2nd place → no trophy	Results → Trophy	Trophy list empty	Empty	None
E03	50th place → no trophy	Results → Trophy	Trophy list empty	Empty	None
E04	3 wins → 3 distinct trophies	Results → Trophy	3 trophies	len=3	None
E05	Trophy title contains race ID	Results → Trophy	"GRAND_PRIX" in title	Contains ID	None

Why these tests matter: Trophies are exclusive rewards for winners. These 5 tests verify the conditional logic (position==1) in the Results → Trophy data path, including multi-win accumulation and title correctness.

3.6 Group F: Missions ↔ Crew Role Check (8 tests)

Call Graph Edge: `MS1: assign_mission()` → `C2: is_role()`

Table 7. Group F — Missions ↔ Crew Role Check

ID	Scenario	Modules	Expected	Actual	Errors
F01	Delivery mission → driver	Crew → Mission	Accepted	Accepted	None
F02	Delivery mission → mechanic	Crew → Mission	Rejected: needs driver	Rejected	None
F03	Rescue mission → strategist	Crew → Mission	Accepted	Accepted	None
F04	Rescue mission → driver	Crew → Mission	Rejected: needs strategist	Rejected	None
F05	Sabotage mission → mechanic	Crew → Mission	Accepted	Accepted	None
F06	Sabotage mission → strategist	Crew → Mission	Rejected: needs mechanic	Rejected	None
F07	Unknown mission type ("Heist")	Mission	Rejected: "Unknown"	Rejected	None
F08	Mission with unregistered member	Crew → Mission	Rejected: no role found	Rejected	None

Why these tests matter: The assignment states *"Missions cannot start if required roles are unavailable."* These 8 tests verify the Mission module's role-checking logic for all 4 mission types (Delivery, Rescue, Sabotage, Repair), covering both success and failure paths.

3.7 Group G: Missions ↔ Inventory Damage (5 tests)

Call Graph Edges: `MS1: assign_mission()` → `I8: is_damaged()` + `I7: set_damage()`

Table 8. Group G — Missions ↔ Inventory Damage

ID	Scenario	Modules	Expected	Actual	Errors
G01	Mechanic repairs damaged car	Crew, Inv → Mission	Accepted, car fixed	Accepted, is_damaged=Fa	Fixed: Missing Inventory dependency in Mission-Module constructor during initial integration.
G02	Repair an undamaged car	Inv → Mission	Rejected: "not damaged"	Rejected	None
G03	Repair without specifying car name	Mission	Rejected: "Must specify"	Rejected	None
G04	Driver tries to repair damaged car	Crew → Mission	Rejected: needs mechanic	Rejected	None
G05	Two damaged cars, repair only one	Inv → Mission	Car1=fixed, Car2=still damaged	Correct	None

Why these tests matter: The assignment states *"If a car is damaged during a race, a mission requiring a mechanic must check for availability before proceeding."* These 5 tests verify the dual-check (role + damage state) and the state mutation (repair) logic.

3.8 Group H: Tuning ↔ Crew + Inventory (8 tests)

Call Graph Edges: `T1: upgrade_car()` → `C2: is_role()` + `I6: use_parts()` + `I2: deduct_cash()` + `I5: add_parts()` *(rollback)*

Table 9. Group H — Tuning ↔ Crew + Inventory

ID	Scenario	Modules	Expected	Actual	Errors
H01	Mechanic + parts + cash → success	Crew, Inv → Tuning	Accepted, cash −$500, parts −1	All correct	None
H02	Driver tries to tune	Crew → Tuning	Rejected: not a mechanic	Rejected	None
H03	No parts available	Inv → Tuning	Rejected, cash untouched	Rejected, cash same	None
H04	Parts OK, insufficient cash → rollback	Inv → Tuning	Rejected, parts restored	Parts=1 (rolled back)	None
H05	Speed upgrade → speed +10	Tuning	speed=10	speed=10	None
H06	Handling upgrade → handling +10	Tuning	handling=10	handling=10	None
H07	3 upgrades → stats stack	Tuning, Inv	speed=20, handling=10	Correct	None
H08	Untuned car → zero stats	Tuning	speed=0, handling=0	Defaults	None

Why these tests matter: Tuning is the most complex 3-way integration — it simultaneously checks Crew (role), Inventory (parts & cash), and records stats. Test H04 specifically verifies the rollback mechanism to prevent partial resource consumption on failure.

3.9 Group I: Sponsors ↔ Inventory (4 tests)

Call Graph Edge: `S2: trigger_win_bonus()` → `I1: add_cash()`

Table 10. Group I — Sponsors ↔ Inventory

ID	Scenario	Modules	Expected	Actual	Errors
I01	Signed sponsor → bonus credits cash	Sponsor → Inv	Cash +$5,000	$10,000→$15	None
I02	No sponsor → no bonus	Sponsor → Inv	Cash unchanged	Same	None
I03	Two sponsors, two races → both paid	Sponsor → Inv	Cash +$4,000 total	$10,000→$14	None
I04	Re-signing overwrites old deal	Sponsor	New bonus=$9,999	$9,999 credited	None

Why these tests matter: Sponsor bonuses represent an additional revenue stream that must flow correctly into the same Inventory cash pool as race prizes. These 4 tests verify the data path and edge cases like missing/overwritten deals.

3.10 Group J: Full Workflow Integration (10 tests)

Call Graph Paths: Multi-module chains spanning 3+ modules

Table 11. Group J — Full Workflow Integration

ID	Scenario	Modules	Expected	Actual	Err.
J01	Register → Race → Win → Cash + Trophy	Reg, Crew, Inv, Race, Res, Trophy	Cash +$10K, trophy earned	All correct	None
J02	Race → Crash → Hire Mechanic → Repair	Reg, Crew, Inv, Race, Res, Mission	Car fixed after repair	is_damaged=	None
J03	Win → Prize + Sponsor Bonus	Reg, Crew, Inv, Race, Res, Sponsor	Cash +$8K ($3K+$5K)	$10,000→$18	None
J04	Tune car → Race with tuned car	Reg, Crew, Inv, Tuning, Race	Tuned, then raced	Both succeeded	None
J05	Tuning depletes cash, winning restores it	Crew, Inv, Tuning, Race, Results	Cash = initial−500+200	Correct	None
J06	3 crew, 3 parallel missions	Reg, Crew, Mission	3 missions active	len=3	None
J07	Crash → Repair → Race again (full cycle)	All core modules	Car reusable after repair	2nd race succeeded	None
J08	Main controller: `run_race_sequence()`	Main, Race, Res, Sponsor	Cash=$13K, trophy earned	Correct	Fixed: Parameter name mismatch (`prize` vs `prize_money`) in `ResultsModule` call.
J09	Main controller: `perform_tuning()`	Main, Tuning, Crew, Inv	Upgrade succeeded	Correct	None
J10	Main controller: `start_mission()`	Main, Mission, Crew	Mission assigned	Correct	None

Why these tests matter: Workflows J01–J07 simulate realistic multi-step operations that a TA would test manually. J08–J10 verify the Main controller correctly delegates to the service modules.

4 Additional Modules (Bonus)

1. **Vehicle Tuning Module**: Upgrades a car's speed or handling when a registered mechanic has enough parts and cash.

2. **Sponsorships Module**: Stores sponsor deals and pays a bonus into Inventory when a sponsored race is won.
3. **Trophy Room Module**: Saves race trophies when a first-place result is recorded.

5 Testing Results Summary

All **65 Integration Test Cases** passed 100% after resolving initial integration bugs. The testing process was instrumental in catching dependency gaps and naming inconsistencies across module boundaries.

5.1 Bugs Detected & Fixed During Integration

1. **Dependency Injection Error**: `MissionModule` was initially initialized without the `InventoryModule` reference, causing `Repair` missions to fail with an `AttributeError`.
2. **Interface Mismatch**: The `StreetRaceManager.run_race_sequence` was passing `prize` while `ResultsModule` expected `prize_money`. This was caught during the first end-to-end integration test.
3. **State Synchronization**: Car damage state was initially not resetting to `False` after a successful `Repair` mission; logic was updated to call `set_damage(False)` upon mission completion.

```
pytest integration/tests/test_integration.py -v
65 passed in 0.12s
```

Final verdict: All 65 integration tests passed and all required module interactions were verified.

6 Exhaustive Integration Audit (Final Proof)

A comprehensive end-to-end simulation was performed to verify the stability of the 9-module cluster.

6.1 Audit Lifecycle Phases

1. **Recruitment**: Successfully registered Alice (Driver), Bob (Mechanic), and Charlie (Strategist).
2. **Asset Management**: Populated garage and warehouse; verified resource availability for tuning.
3. **Competition Flow**: Alice won a high-stakes race; verified automatic **Trophy Unlock** and **Sponsorship Bonus** ($35,000 total gain).
4. **Resilience**: Simulated a vehicle crash; verified that only a **Mechanic** could perform the **Repair Mission**.
5. **Tuning**: Bob successfully upgraded the Skyline GTR using inventory engine modules.
6. **Logistics**: Charlie executed a rescue mission requiring strategical clearance.

Final System State: 100% stability. Final financial balance: **$44,500**.